

Module 3: Operators and Expressions in PL/SQL

Duration

2–3 Hours (Theory + Hands-on Practice)

Learning Outcomes

After completing this module, students will be able to:

- Understand different types of operators in PL/SQL.
 - Use arithmetic operators to perform mathematical calculations.
 - Apply comparison operators to compare values.
 - Evaluate logical expressions using logical operators.
 - Concatenate strings using string operators.
 - Understand operator precedence.
 - Write PL/SQL programs using expressions.
 - Develop simple applications involving calculations.
-

3.1 Introduction to Operators

What is an Operator?

An **operator** is a symbol that instructs the PL/SQL compiler to perform a specific operation on one or more operands.

An **operand** may be a variable, constant, literal, or expression.

Example

10 + 20

Here,

- 10 → Operand
- 20 → Operand
- - → Operator

Result = 30

Why Do We Need Operators?

Almost every database application performs calculations and comparisons.

Examples:

- Calculate employee salary.
- Calculate student percentage.
- Compare salaries.
- Determine whether a student has passed or failed.
- Calculate GST.
- Generate payroll.
- Calculate bank interest.

Without operators, such tasks would not be possible.

Types of Operators in PL/SQL

PL/SQL provides the following categories of operators:

Operator Type	Purpose
Arithmetic Operators	Perform mathematical calculations
Relational Operators	Compare values
Logical Operators	Combine multiple conditions
Assignment Operator	Assign values to variables
String Operator	Concatenate strings
Special Operators	BETWEEN, IN, LIKE, IS NULL (used mainly in SQL statements)

3.2 Arithmetic Operators

Arithmetic operators perform mathematical operations.

Operator	Description	Example
+	Addition	10 + 20
-	Subtraction	20 - 5
*	Multiplication	5 * 8
/	Division	20 / 5
**	Exponentiation	2 ** 3

Example 1: Arithmetic Operations

SET SERVEROUTPUT ON;

```
DECLARE
    num1 NUMBER := 20;
    num2 NUMBER := 10;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Addition = ' || (num1 + num2));
    DBMS_OUTPUT.PUT_LINE('Subtraction = ' || (num1 - num2));
    DBMS_OUTPUT.PUT_LINE('Multiplication = ' || (num1 * num2));
    DBMS_OUTPUT.PUT_LINE('Division = ' || (num1 / num2));
END;
/
```

Output

Addition = 30
Subtraction = 10
Multiplication = 200
Division = 2

Example 2: Area of a Rectangle

Formula:

Area = Length × Breadth

```
DECLARE
    length NUMBER := 15;
    breadth NUMBER := 8;
    area NUMBER;
BEGIN
    area := length * breadth;

    DBMS_OUTPUT.PUT_LINE('Area = ' || area);
END;
/
```

Output
Area = 120

Example 3: Area of a Circle

Formula

Area = $\pi \times r^2$

```
DECLARE
    radius NUMBER := 7;
    area NUMBER;
    pi CONSTANT NUMBER := 3.14159;
BEGIN
    area := pi * radius * radius;

    DBMS_OUTPUT.PUT_LINE('Area = ' || area);
END;
```

/

3.3 Relational Operators

Relational operators compare two values.
The result is always **TRUE** or **FALSE**.

Operator Meaning

=	Equal
!=	Not Equal
<>	Not Equal
>	Greater Than
<	Less Than
>=	Greater Than or Equal
<=	Less Than or Equal

Example

```
DECLARE
    salary NUMBER := 45000;
BEGIN
    IF salary > 30000 THEN
        DBMS_OUTPUT.PUT_LINE('High Salary');
    END IF;
END;
/
Output
High Salary
```

Example: Compare Two Numbers

```
DECLARE
    a NUMBER := 25;
    b NUMBER := 40;
BEGIN
    IF a < b THEN
        DBMS_OUTPUT.PUT_LINE('B is Greater');
    END IF;
END;
/
```

3.4 Logical Operators

Logical operators combine multiple conditions.

Operator Meaning

AND	Both conditions must be TRUE
OR	At least one condition must be TRUE
NOT	Reverses the condition

Example: AND Operator

```
DECLARE
    marks NUMBER := 85;
    attendance NUMBER := 90;
BEGIN
    IF marks >= 40 AND attendance >= 75 THEN
        DBMS_OUTPUT.PUT_LINE('Eligible');
    END IF;
```

```
END;  
/  
Output  
Eligible
```

Example: OR Operator

```
DECLARE  
    sports BOOLEAN := TRUE;  
    cultural BOOLEAN := FALSE;  
BEGIN  
    IF sports OR cultural THEN  
        DBMS_OUTPUT.PUT_LINE('Student Selected');  
    END IF;  
END;  
/
```

Example: NOT Operator

```
DECLARE  
    is_holiday BOOLEAN := FALSE;  
BEGIN  
    IF NOT is_holiday THEN  
        DBMS_OUTPUT.PUT_LINE('College is Open');  
    END IF;  
END;  
/
```

Truth Table

A	B	A AND B	A OR B
TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE
FALSE	TRUE	FALSE	TRUE
FALSE	FALSE	FALSE	FALSE

3.5 Assignment Operator

PL/SQL uses
:=
to assign values.
Example
salary := 50000;

3.6 String Operator

PL/SQL uses
||
for string concatenation.

Example

```
DECLARE  
    first_name VARCHAR2(20) := 'Rahul';  
    last_name VARCHAR2(20) := 'Sharma';  
BEGIN  
    DBMS_OUTPUT.PUT_LINE(first_name || ' ' || last_name);  
END;  
/  
Output
```

Example: Student Details

```
DECLARE
    name VARCHAR2(20) := 'Amit';
    course VARCHAR2(20) := 'B.Tech';
BEGIN
    DBMS_OUTPUT.PUT_LINE('Student : ' || name);
    DBMS_OUTPUT.PUT_LINE('Course : ' || course);
END;
/
```

3.7 Operator Precedence

When an expression contains multiple operators, PL/SQL follows precedence rules.

Priority Operator

Highest **
*, /
+, -
Lowest =, <>, >, <

Example

```
DECLARE
    result NUMBER;
BEGIN
    result := 10 + 5 * 2;

    DBMS_OUTPUT.PUT_LINE(result);
END;
/
```

Output
20

Explanation
 $5 \times 2 = 10$
 $10 + 10 = 20$

3.8 Expressions

An expression is a combination of variables, constants, operators, and functions that produces a value.

Examples

salary + bonus

marks / 5

basic + hra + da

name || surname

Complete Program: Employee Salary

```
DECLARE
    basic NUMBER := 40000;
    hra NUMBER;
    da NUMBER;
    gross NUMBER;
BEGIN
```

```
hra := basic * 0.20;
da := basic * 0.10;

gross := basic + hra + da;

DBMS_OUTPUT.PUT_LINE('Basic Salary : ' || basic);
DBMS_OUTPUT.PUT_LINE('HRA : ' || hra);
DBMS_OUTPUT.PUT_LINE('DA : ' || da);
DBMS_OUTPUT.PUT_LINE('Gross Salary : ' || gross);
END;
/
```

Complete Program: Student Percentage

```
DECLARE
    dbms NUMBER := 85;
    java NUMBER := 92;
    python NUMBER := 88;
    total NUMBER;
    percentage NUMBER;
BEGIN
    total := dbms + java + python;

    percentage := total / 3;

    DBMS_OUTPUT.PUT_LINE('Total Marks : ' || total);
    DBMS_OUTPUT.PUT_LINE('Percentage : ' || ROUND(percentage,2));
END;
/
```

Output
Total Marks : 265
Percentage : 88.33

Real-World Applications

Operators are used in:

- Payroll Management Systems
 - Banking Applications
 - Student Result Processing
 - GST and Tax Calculations
 - Hospital Billing Systems
 - E-commerce Discount Calculations
 - Loan and EMI Calculations
 - Attendance and Payroll Automation
-

Common Mistakes by Beginners

1. Using = instead of := for variable assignment.
 2. Forgetting to initialize variables before using them.
 3. Confusing string concatenation (||) with arithmetic addition (+).
 4. Ignoring operator precedence in complex expressions.
 5. Performing division without considering zero values, which can raise exceptions.
-

Laboratory Exercises

1. Write a program to calculate the area and perimeter of a rectangle.
2. Calculate the area and circumference of a circle.
3. Write a program to calculate the simple and compound interest.
4. Calculate the gross salary using Basic Salary, HRA, DA, and other allowances.

5. Write a program to calculate the percentage of five subjects.
 6. Compare the salaries of two employees and display the higher salary.
 7. Demonstrate the use of AND, OR, and NOT operators with suitable examples.
 8. Concatenate a student's first name, middle name, and last name into a single string.
 9. Write a program to evaluate an arithmetic expression using multiple operators and verify operator precedence.
 10. Develop a basic electricity bill calculator using arithmetic expressions.
-

Viva/Interview Questions

1. What is an operator in PL/SQL?
2. Differentiate between arithmetic and relational operators.
3. Explain the purpose of logical operators with examples.
4. What is the difference between the assignment operator (:=) and the equality operator (=)?
5. How is string concatenation performed in PL/SQL?
6. What is an expression in PL/SQL?
7. Explain operator precedence with an example.
8. Which operators return Boolean values?
9. Why are logical operators important in database applications?
10. Give two real-world scenarios where arithmetic and relational operators are commonly used in PL/SQL.